★ System Design Handwritten Notes ★

What is System Design?

→ System Design is the process of designing large scale, reliable, scalable & efficient systems to solve real world problems.

Why System Design?

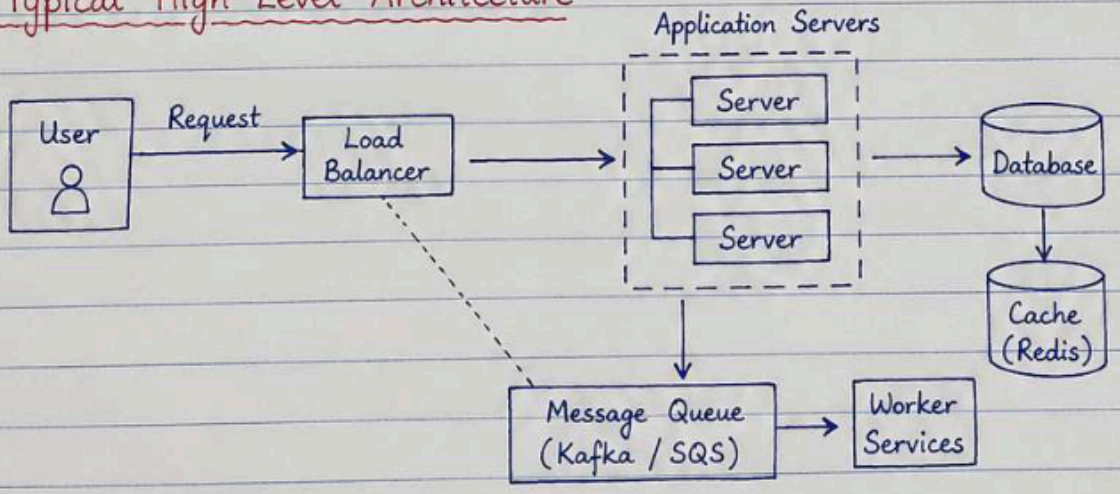
→ Helps in building scalable systems.
→ Handles high traffic and large data.
→ Improves availability, reliability and performance.
→ Helps in making the system fault tolerant.

This Notebook Covers

① Fundamentals of System Design
② High Level Design (HLD)
③ Core Components & Their Roles
④ Scalability, Availability, Reliability
⑤ Databases, Caching, Load Balancing
⑥ Real World System Designs
⑦ Trade-offs and Best Practices



Typical High Level Architecture



Key Points to Remember

- Always think about scale from the beginning.
- Understand the requirements and constraints.
- Keep the design simple, modular and extensible.
- Choose the right technology for the right problem.

Happy Learning! ☺

1. How a Request Travels Through the Internet ★

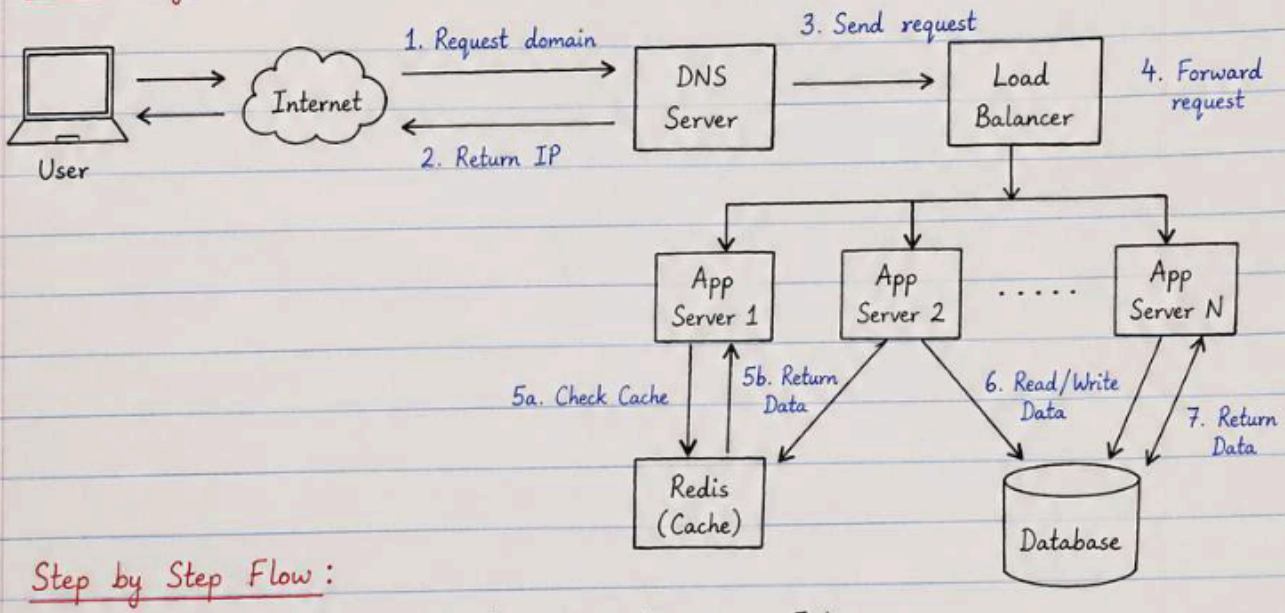
When you type a URL in your browser and press Enter, a lot of things happen behind the scenes.

The main components involved are :

1. DNS (Domain Name System) - Converts domain name to IP address.
2. Load Balancer - Distributes incoming requests to multiple servers.
3. Application Server - Processes the request and contains business logic.
4. Redis (Cache) - Stores frequently accessed data for faster response.
5. Database - Stores persistent data.

Let's see the complete flow.

Flow Diagram :



Step by Step Flow :

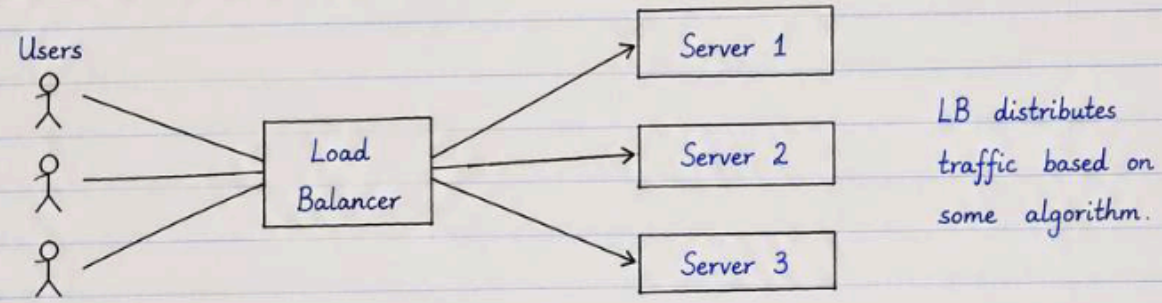
1. User enters a URL in the browser and presses Enter.
2. Browser sends a request to DNS server to resolve the domain name.
3. DNS returns the IP address of the server (or Load Balancer).
4. Request is sent to the Load Balancer.
5. Load Balancer selects an Application Server and forwards the request.
6. Application Server checks Redis (cache) for the required data.
   - If data is found in cache (hit), it returns the data.
   - If not found (miss), it fetches data from Database.
7. Database returns the data to Application Server.
8. Application Server sends the response back to the user through Load Balancer, Internet.

2. Load Balancer ☆

Why Load Balancer ?

- Distributes incoming traffic across multiple servers.
- Prevents any single server from getting overloaded.
- Improves availability and reliability.
- Helps in scaling applications.
- Provides Health checks to route traffic only to healthy servers.

How Load Balancer Works ?

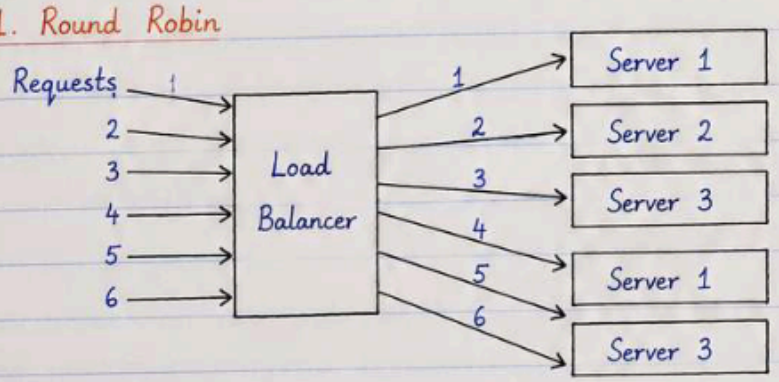


LB distributes traffic based on some algorithm.

Load Balancing Algorithms

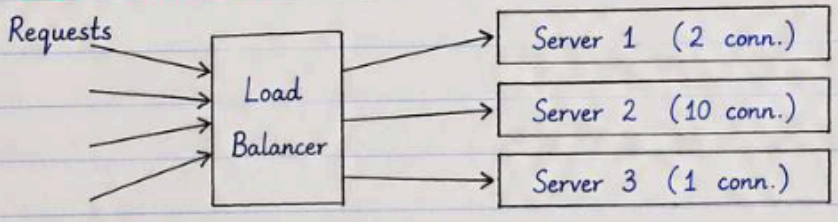
1. Round Robin – Requests are distributed one by one in circular order.
2. Least Connections – Request is sent to the server with least number of active connections.
3. IP Hash – Same IP always goes to the same server.
4. Weighted Round Robin – Servers have different weights, traffic is distributed based on weight.

1. Round Robin



Order: 1 → S1
2 → S2
3 → S3
4 → S1
5 → S2
6 → S3
... and so on

2. Least Connections



Request will go to Server 3 because it has least connections.

Health Checks

- Load Balancer periodically checks each server (e.g. every 10 sec).
- If a server is down / not responding, it is marked as unhealthy.
- Traffic is not sent to unhealthy servers.
- Once the server is healthy again, it starts receiving traffic.

3. Application Server ☆

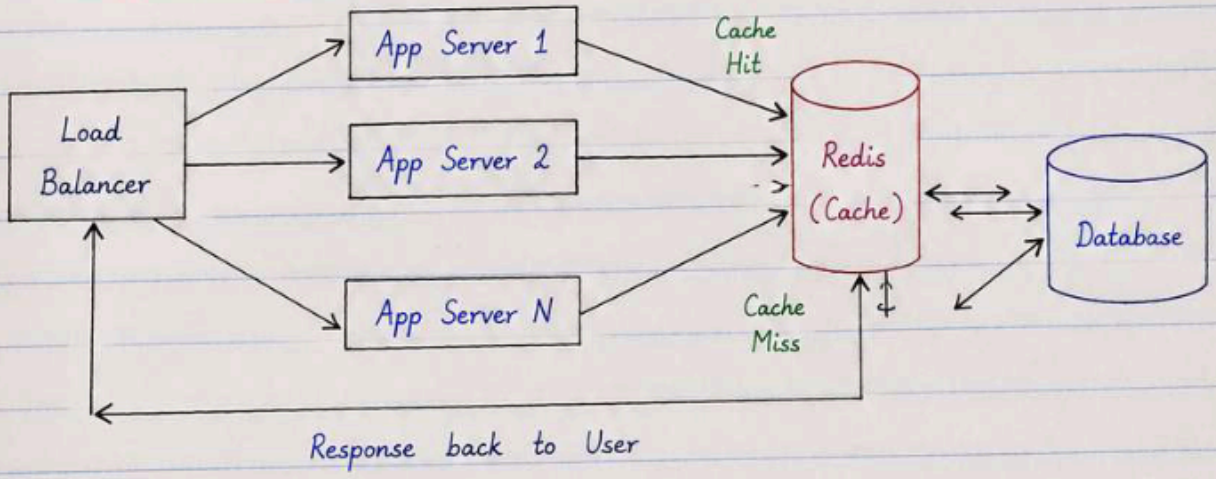
What is Application Server ?

Application Server processes the request, contains the business logic of the application and interacts with Cache / Database to get or save data.

Responsibilities :

- Handle client requests.
- Execute business logic.
- Interact with Cache (Redis) for faster access.
- Read from / Write to Database if not in cache.
- Return the response back to Load Balancer.

How Application Server Works ?



Flow in Detail :

1. Load Balancer forwards the request to one of the Application Servers.
2. Application Server checks Redis (Cache) for the required data.
    - If data is found in cache (Hit) → Use it.
    - If not found (Miss) → Fetch from Database.
3. If fetched from Database, store it in Redis for future requests.
4. Process the request and generate the response.
5. Send the response back to Load Balancer.
6. Load Balancer returns the response to the user.

Benefits :

- Reduces database load.
- Faster response due to caching.
- Improves scalability.
- Better user experience.

4. Redis (Cache) ☆

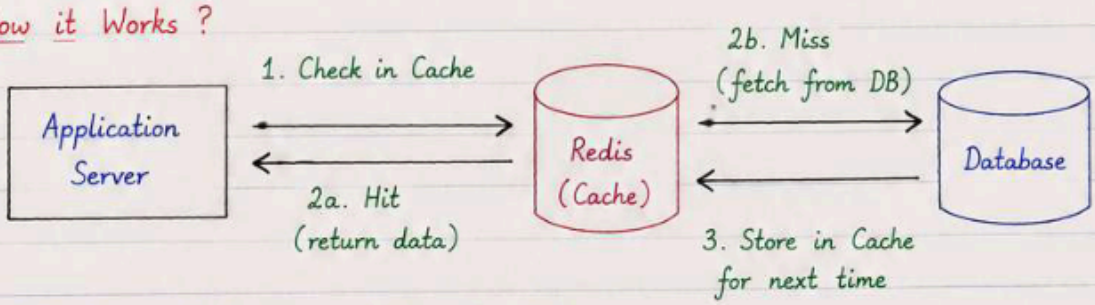
What is Redis ?

Redis is an in-memory data structure store, used as a cache, message broker and database. It stores data in key-value format and is very fast.

Why use Redis (Cache) ?

- Stores frequently accessed data in memory.
- Reduces load on Database.
- Faster response time.
- Supports various data structures (String, Hash, List, Set, Sorted Set etc.).
- Can be used for pub/sub, session store, rate limiting, leaderboards, etc.

How it Works ?



- Application first checks Redis for data.
- If data is found (Hit) → return to application.
- If not found (Miss) → fetch from Database.
- Store the data in Redis for future requests and return to application.

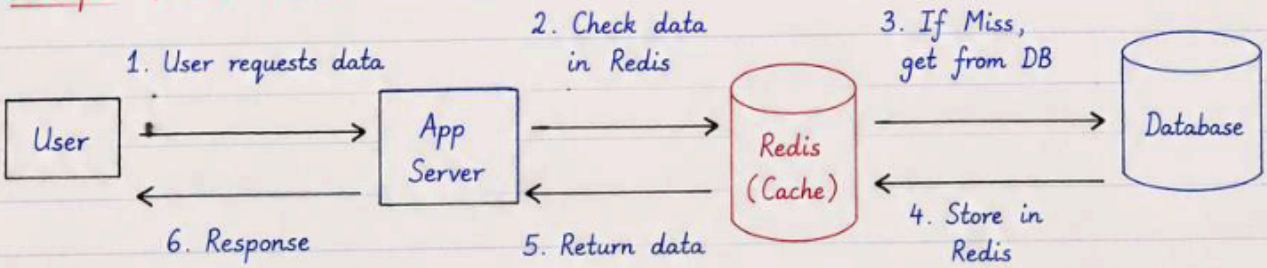
Common Data Structures in Redis

- String → key : value
- Hash → key : field : value
- List → key : [a, b, c]
- Set → key : {a, b, c}
- Sorted Set → key : {a:1, b:2, c:3}

Use Cases

- Caching
- Session store
- Rate limiting
- Leaderboards
- Messaging (Pub/Sub)
- Counters, Tokens, Locks

Example (Cache Flow)



Note : Redis being in-memory is extremely fast compared to Database. It is usually used along with Database, not as a replacement.

3. Caching (Redis) ☆

Cache Concepts in Detail

1. Cache Hit

- When the requested data is found in Redis cache.
  ⇒ Fast response, no need to go to Database.

2. Cache Miss

- When the requested data is not found in Redis cache.
  ⇒ Data is fetched from Database and stored in Redis for future requests.

3. TTL (Time To Live)

- Expiry time for a key in Redis.
- After TTL expires, key will be automatically removed.
- Helps to keep data fresh and save memory.

Syntax : SET key value EX 60 → (expires in 60 sec)

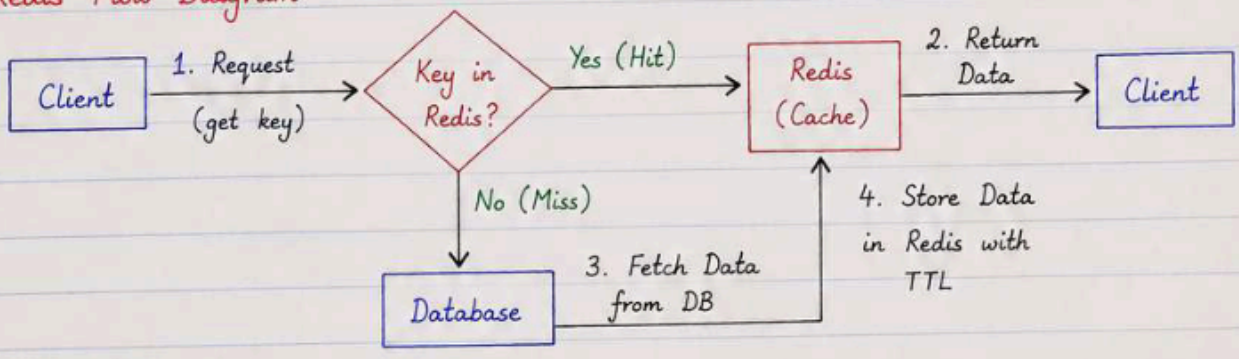
4. Eviction

- When Redis memory is full and new data needs to be stored.
- Redis removes some old keys based on eviction policy.

Common Eviction Policies :

1. noeviction → don't remove anything, just return error.
2. allkeys-lru → remove least recently used keys.
3. allkeys-random → remove random keys.
4. volatile-lru → remove least recently used keys with TTL.
5. volatile-ttl → remove keys with shortest TTL.

Redis Flow Diagram



Example Flow :

1. Client requests data for a key.
2. Redis checks if key exists.
   - If exists (Hit) → return data to client.
   - If not (Miss) → fetch from DB.
3. Store the fetched data in Redis with TTL.
4. Return data to client.

Note : Caching reduces database load and improves response time.
Redis being in-memory, is extremely fast.

4. Database Scaling ☆

1. Vertical Scaling (Scale Up)

- Increase the power of the existing server.
- Add more CPU, RAM, SSD etc.
- Easy but has limit.

2. Horizontal Scaling (Scale Out)

- Add more servers to handle the load.
- Distribute the load among multiple servers.
- More complex but highly scalable.

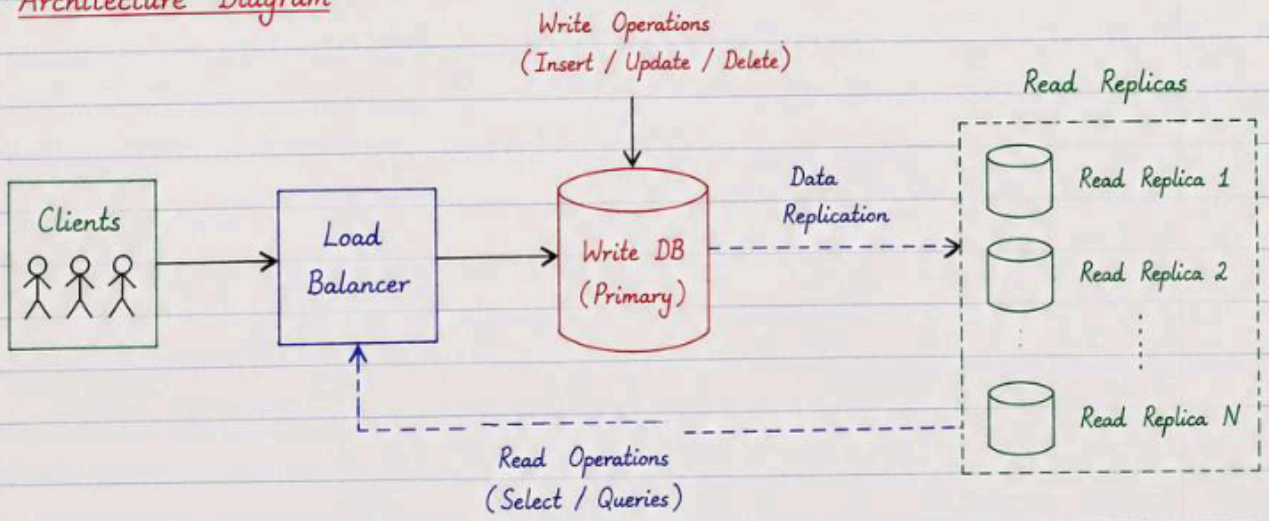
3. Read Replica

- Copy of the main database.
- Used for read operations.
- Helps in reducing load on Write DB.

4. Write Database (Primary)

- Main database where all write operations happen (Insert, Update, Delete).
- Ensures data consistency.

Architecture Diagram



How it Works

1. Client sends request.
2. Load Balancer forwards write requests to Primary DB.
3. Primary DB handles writes and replicates data to Read Replicas.
4. Load Balancer forwards read requests to Read Replicas.
5. Read Replicas respond to client.

Notes

- Improves performance and scalability.
- Write goes to Primary only.
- Read can go to any Replica.
- Replication can be async or sync.

5. API Gateway

API Gateway is a single entry point for all client requests.
It sits between client and multiple backend services.

1. Authentication

- Validates the user / client.
- Checks token, API key, or credentials.
- If valid → forward request.
- If invalid → return error.

2. Routing

- Decides which service should handle the request.
- Based on path, headers, or parameters.
- Forwards request to the correct service.

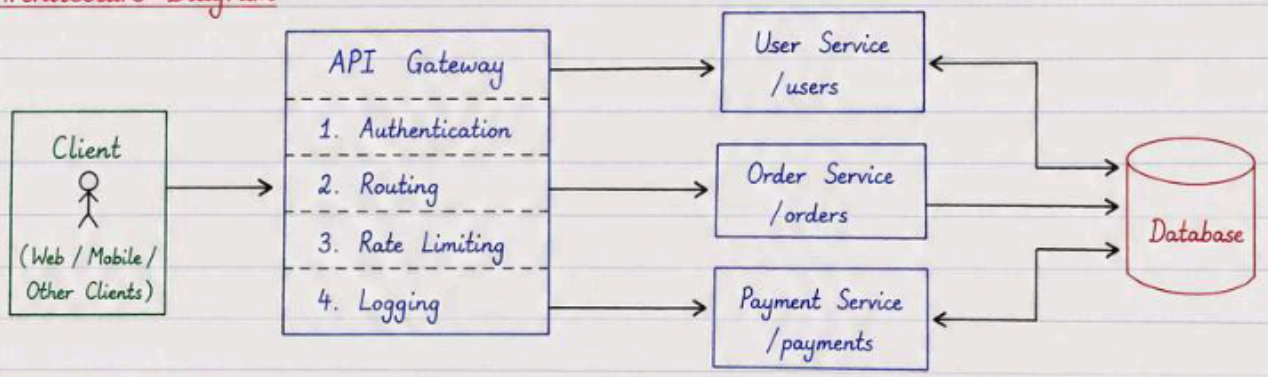
3. Rate Limiting

- Limits number of requests from a client.
- Prevents abuse and protects services.
- Example: 100 requests / minute per user.

4. Logging

- Logs every request and response.
- Helps in monitoring and debugging.
- Stores metadata like time, status code, latency etc.

Architecture Diagram



How it Works (Flow)

1. Client sends request to API Gateway.
2. Gateway authenticates the client.
↓
3. Gateway checks rate limit.
↓
4. Gateway routes the request to the appropriate service.
↓
5. Service processes the request and sends response.
↓
6. Gateway logs the request & response.
↓
7. Gateway sends response back to client.

Notes

- Simplifies client interaction.
- Centralized security, monitoring and routing.
- Helps in scalability and maintainability.

6. Microservices ☆

1. Monolith vs Microservices

Monolith	Microservices
• Single codebase for entire application.	• Application is broken into small services.
• All modules are tightly coupled.	• Services are loosely coupled.
• One deployment for the whole app.	• Each service can be deployed independently.
• Scalability is difficult.	• Easy to scale individual services.
• Failure in one module can bring down the whole system.	• Failure in one service does not affect other services.

2. Advantages

- Independent deployment.
- Scalability → scale only what you need.
- Technology freedom → different services can use different tech stack.
- Fault isolation → failure in one service won't impact others.
- Faster development and delivery.

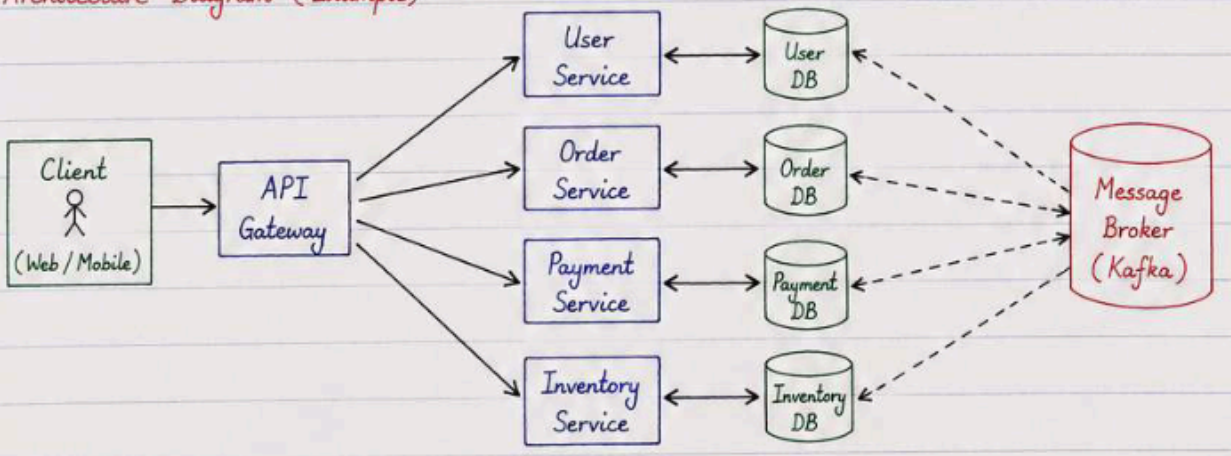
3. Challenges

- More complex to design, develop and test.
- Distributed system → network latency, failures.
- Data consistency across services.
- Monitoring and debugging is hard.
- Requires DevOps / automation.

4. Communication

- Synchronous → REST APIs, gRPC
- Asynchronous → Message Queue (Kafka, RabbitMQ)
- Events / Pub-Sub model for loose coupling.

Architecture Diagram (Example)



Summary

Microservices improve scalability, flexibility and fault isolation. but add complexity to the system.

7. Message Queue (Kafka / RabbitMQ) ☆

Message Queue is used for asynchronous communication between services / applications.

1. Producer

- Produces / sends messages to the queue (topic).
- Does not wait for the consumer.

2. Queue (Broker)

- Stores messages until consumer reads them.
- Ensures delivery of messages.

3. Consumer

- Consumes / reads messages from the queue.
- Processes the message.

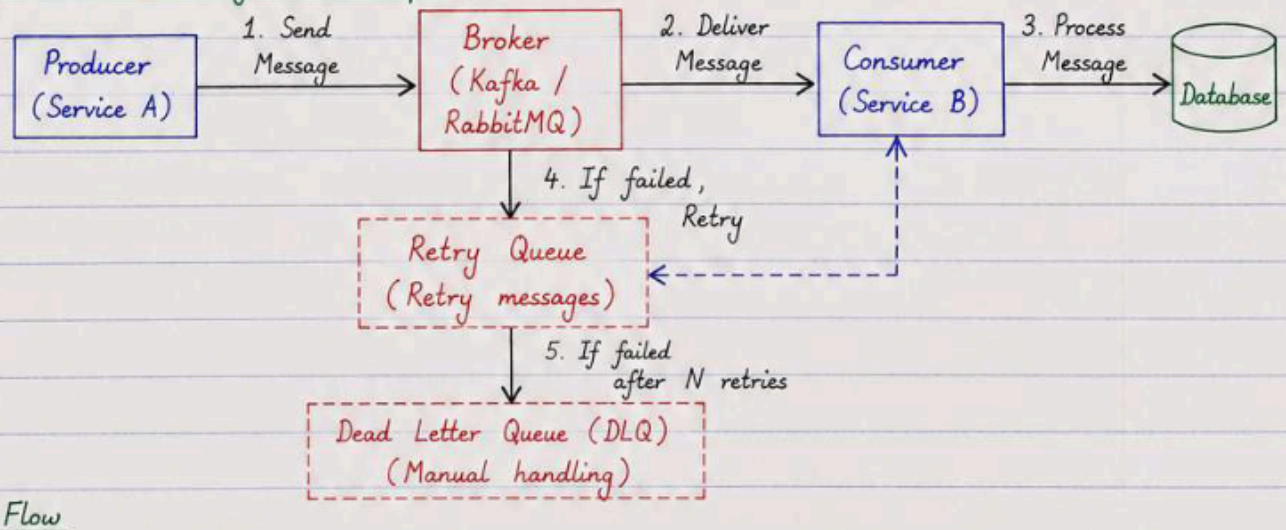
4. Retry Mechanism

- If processing fails, message can be retried.
- Prevents data loss.

5. Dead Letter Queue (DLQ)

- Messages that fail even after multiple retries are moved to DLQ.
- Helps in debugging and manual handling.

Architecture Diagram (Example)



Flow

1. Producer sends message to Broker.
2. Broker stores message and delivers to Consumer.
3. Consumer processes the message.
4. If processing fails → message goes to Retry Queue.
5. After N retries, if still fails → message goes to DLQ.

Examples

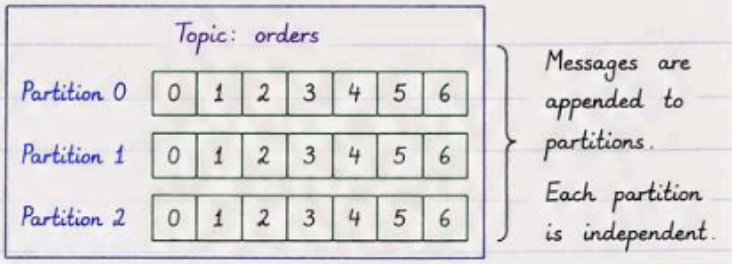
- Order Service places order → sends message to "order-created" topic.
- Payment Service consumes message → processes payment.
- If payment fails → retry.
- If still fails → moved to DLQ for manual check.

7. Message Queue (Kafka / RabbitMQ) - More Concepts

Message Queues help in asynchronous communication, decoupling services, and handling high traffic reliably.

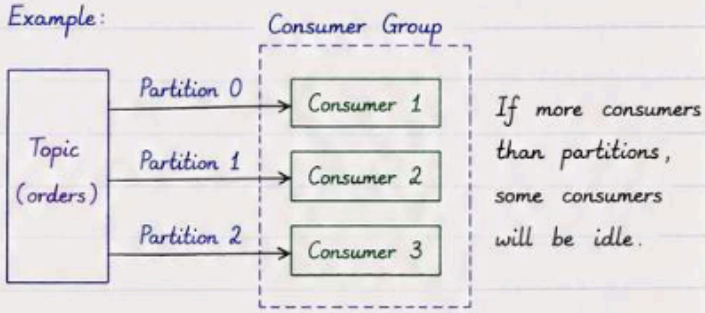
1. Partition (Kafka)

- Topic is divided into multiple partitions.
- Messages in a partition are ordered.
- Partitions allow parallel processing by consumers.
- Each partition has its own offset.



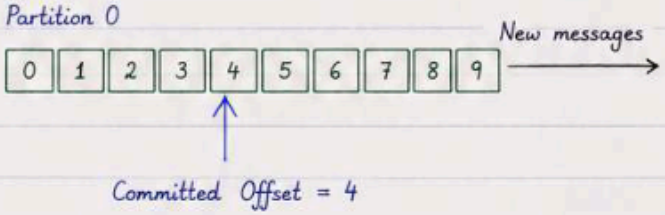
2. Consumer Group

- Consumers work in a group to consume messages.
- Each partition can be consumed by only one consumer in a group.
- This allows parallel consumption.



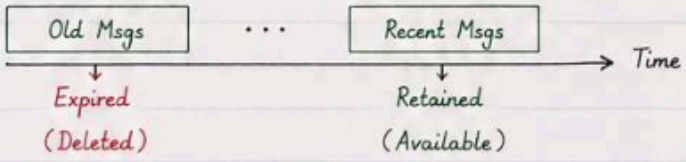
3. Offset

- Offset is a unique ID for each message in a partition.
- Consumer commits offset after processing.
- On restart, consumer reads from last committed offset.



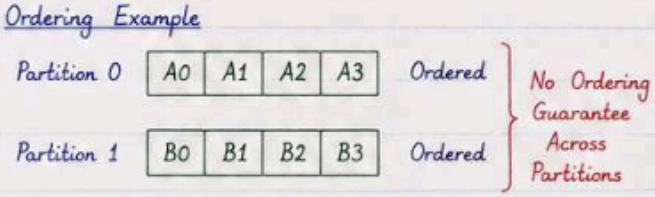
4. Retention Policy

- Determines how long messages are kept.
- Time based retention (e.g. 7 days).
- Size based retention (e.g. 10 GB).
- After retention, old messages are deleted.



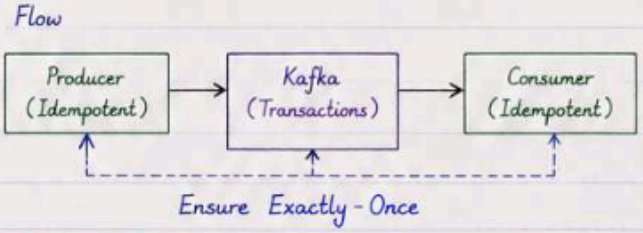
5. Ordering Guarantee

- Messages are ordered within a partition.
- No ordering guarantee across partitions.
- Use single partition if strict ordering is required.



6. Exactly-Once Processing (Kafka)

- Ensures a message is processed exactly once.
- Requires idempotent producer + transaction.
- Prevents duplicates even after failures.



7. Use Cases

- Order processing.
- Event driven communication.
- Log aggregation.
- Real-time analytics.
- Decoupling microservices.

8. Best Practices

- Choose proper partition count.
- Monitor lag and consumer health.
- Set appropriate retention.
- Handle failures with retry and DLQ.
- Use meaningful topic and consumer group names.

8. CDN (Content Delivery Network)

A CDN is a distributed network of servers that delivers static content to users from a location closer to them.

1. Static Content

- CDN is best for static content.
- Examples: Images, CSS, JS, Videos, PDF, Fonts, etc.
- Dynamic content usually goes to origin server.

2. Edge Servers

- CDN has servers (PoPs) in multiple regions.
- These edge servers are closer to users.
- Content is cached at edges for faster delivery.

3. Cache

- Content is stored in cache at edge servers.
- If requested content is in cache → cache hit (fast).
- If not → cache miss → fetched from origin.

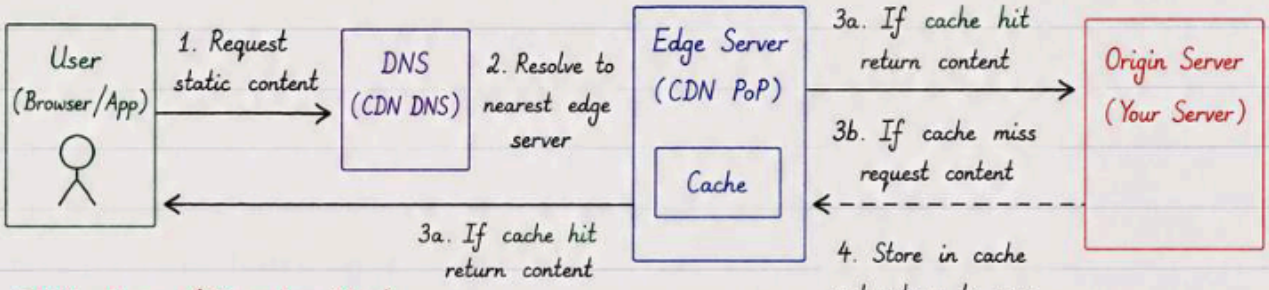
4. Latency

- Serving content from nearest edge redwces network distance.
- Lower latency → better user experience.
- Important for global applications.

5. Benefits

- Faster content delivery
- Reduced load on origin server
- Improved availability & reliability
- Better user experience
- Cost savings on bandwidth

CDN Architecture (Example)



CDN Flow (Step-by-Step)

1. User requests static content (e.g., image, JS file).
2. DNS routes the request to nearest CDN edge server.
3. Edge server checks cache.
   a. If cache hit → content served from edge (fast).
   b. If cache miss → edge server fetches from origin.
4. Edge server stores content in cache.
5. Content delivered to user.

Cache Hit

- Content found in cache at edge.
- Served immediately.
- Low latency.
- Less load on origin server.

Cache Miss

- Content not in cache.
- Edge server requests from origin.
- Stored in cache.
- Then delivered to user.

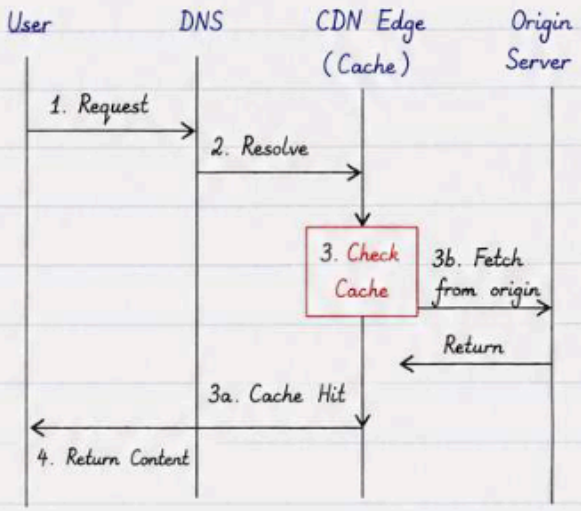
Real World Examples

- User in India requests a website image.
- Indian edge server (CDN PoP) serves it from cache → fast.
- If not in cache → fetched from origin server → stored → served.

8. CDN (Content Delivery Network) - More Details

1. How CDN Works (Detailed Flow)

1. User requests content (e.g., image, video).
2. DNS resolves to the nearest CDN edge server.
3. Edge server checks if content exists in cache.
   - If yes (cache hit) → content is returned to user.
   - If no (cache miss) → edge server fetches from origin server.
4. Content is stored in cache at edge for future requests.
5. Content is delivered to the user.



2. Types of Content Delivered by CDN

- Static files → HTML, CSS, JS, Images, Fonts, PDFs
- Streaming media → Videos, Live streams
- Software / updates → App updates, Patches
- API responses (cacheable) → Public data

3. CDN Components

- DNS → routes user to nearest edge.
- Edge Servers (PoPs) → cache + deliver content.
- Origin Server → source of content.
- Cache → stores content at edge.
- Control Plane → manages config, cache rules.

4. CDN Caching Basics

- Cache stores copies of content at edge.
- When content is requested, CDN serves from cache if available.
- If not available or expired, it fetches from origin.
- This reduces load on origin server and improves speed.

5. Cache Behavior

Scenario	What Happens	Result
Cache Hit	Content found in cache.	Fast response, low latency.
Cache Miss	Content not in cache. Fetched from origin.	Higher latency, origin load increases.
Cache Expired	Content present but expired. Fetched again from origin.	Fresh content delivered, cache updated.
Cache Refresh	Content still valid, but refreshed in background (if supported).	Seamless for user.

6. Common CDN Use Cases

- Website performance improvement.
- Global content delivery.
- Video streaming & live events.
- Game updates & downloads.
- API acceleration (for cacheable APIs).

7. Key Points to Remember

- CDN does not replace origin server.
- It only caches and delivers content.
- Proper cache rules and TTL are important.
- Helps in scalability, reliability, and speed.

9. CAP Theorem

CAP theorem states that a distributed system can guarantee at most two out of the following three properties at the same time.

1. Consistency (C)

- All nodes see the same data at the same time.
- Once data is written, all reads return the latest value.

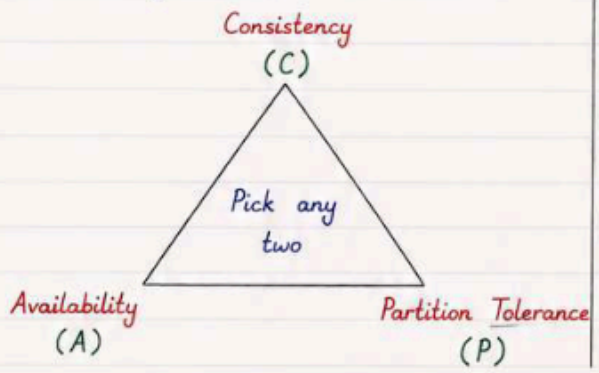
2. Availability (A)

- Every request gets a response (no matter what).
- System is always up and running.

3. Partition Tolerance (P)

- System continues to work even if there is a network partition (some nodes can't talk to others).
- No data is lost.

CAP Triangle



Key Idea

- In case of a network partition (P is needed), you have to choose between Consistency (C) and Availability (A).
- You cannot have all three (C + A + P) together.

CAP Thoices (Examples)

CP (Consistency + Partition Tolerance)	AP (Availability + Partition Tolerance)	CA (Consistency + Availability)
• Consistency is more important. • System may not be available during partition. • Example: HBase, MongoDB, Zoookeeper	• Availability is more important. • System remains available during partition. • Data may be inconsistent temporarily. • Example: Cassandra, DynamoDB, CouchDB	• No partition tolerance. • Works only in single node / reliable network. • Example: MySQL (single node), PostgreSQL (single node)

When does Partition happen?

- Network failure
- Datacenter failure
- Message loss / delay
- Nodes become unreachable

Notes

- Most real-world systems choose either CP or AP.
- It depends on the use case.
- You cannot avoid partitions in distributed systems.
- So, design for partitions and choose the right trade-off.

Real World Examples

System	CAP Choice	Why?
Cassandra	AP	High availability, can handle partitions, may return stale data.
MongoDB	CP	Prefers consistency, may sacrifice availability during partition.
HBase	CP	Built on strong consistency, partitions may block operations.
Redis (Cluster)	AP	High availability is preferred, eventual consistency.
RDBMS (Single Node)	CA	Single server, no partition tolerance needed.

10. Database Sharding

Sharding is a technique to horizontally partition data across multiple databases (shards) to improve scalability and performance.

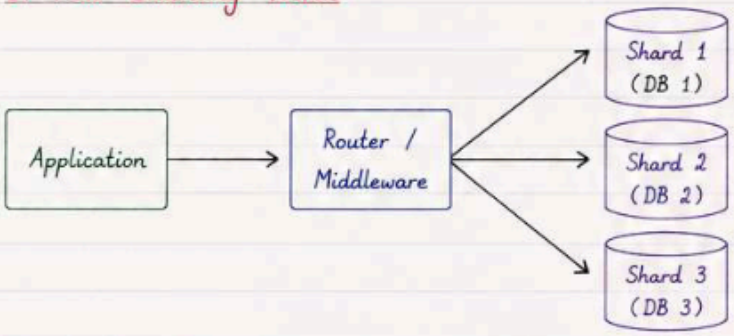
1. Why Sharding?

- Handle large amount of data.
- Distribute read/write load.
- Avoid single database bottleneck.
- Improve availability.

2. Key Terms

- Shard: A single database instance.
- Sharding Key: Column used to decide which shard will store the data.
- Router / Middleware: Decides the correct shard for a query.

3. How Sharding Works



- Application sends query.
- Router uses sharding key to find the shard.
- Query is executed on that shard.

4. Sharding Strategies

Strategy	How it Works	Example	Pros	Cons
1. Range Based	Data is divided based on a range of values.	User ID 1-1000 → Shard 1 1001-2000 → Shard 2 2001-3000 → Shard 3	• Simple to implement • Good for ordered data	• Uneven data distribution
2. Hash Based	Hash function decides the shard.	hash(user_id) % 3 → shard number	• Even data distribution • Easy to scale	• Hard to do range queries
3. Directory Based	Central directory (mapping) keeps track of shards.	Lookup in directory to find shard.	• Flexible • Supports any sharding key	• Extra overhead for directory
4. Geo Based	Data is stored based on geographical location.	US users → Shard 1 EU users → Shard 2 Asia users → Shard 3	• Low latency for users • Legal / compliance friendly	• More complex to manage

5. Example (Hash Based Sharding)

Sharding Key : user_id
Number of Shards : 3

user_id = 101 → hash(101) % 3 = 2 → Shard 2
user_id = 102 → hash(102) % 3 = 0 → Shard 0
user_id = 103 → hash(103) % 3 = 1 → Shard 1
user_id = 104 → hash(104) % 3 = 2 → Shard 2

6. Challenges

- Cross-shard joins are difficult.
- Re-sharding is complex.
- Transactions across shards are hard.
- Data distribution can become uneven.
- More operational overhead.

7. Best Practices

- Choose the right sharding key.
- Start with a simple strategy (e.g., hash based).
- Monitor data distribution and shard performance.
- Plan for resharding from the beginning.
- Keep shard size balanced.

8. When to Use Sharding?

- When a single database can't handle the data.
- When there is high read/write load.
- When you need horizontal scalability.
- When low latency is critical.

11. URL Shortener System Design

A URL shortener converts a long URL into a short, unique URL that redirects to the original URL.

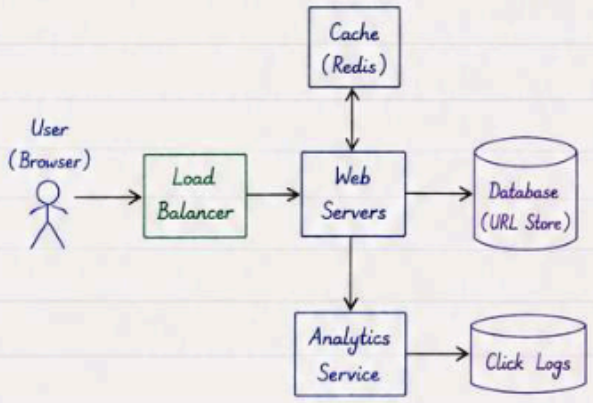
1. Requirements

- Shorten long URLs.
- Redirect short URL to original URL.
- Custom short URLs (optional).
- High availability and low latency.
- Analytics - track clicks.
- Handle large scale.

2. Basic API

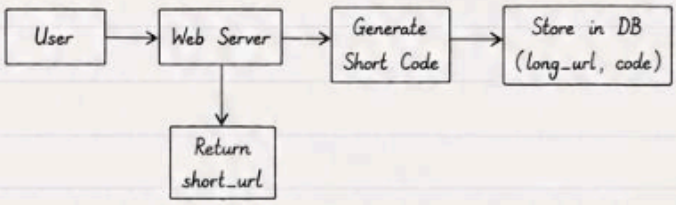
- POST /api/shorten
  Request : { long_url }
  Response : { short_url }
- GET /{short_code}
  Redirects to long_url

3. High Level Design

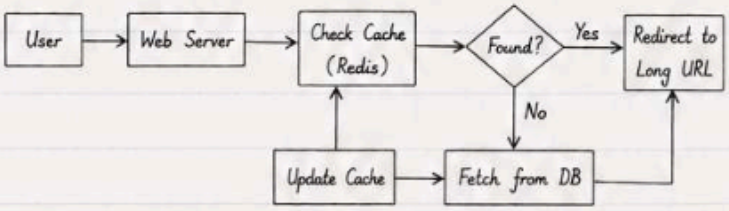


4. Flow

a) Shorten URL



b) Access Short URL



5. Short Code Generation

- Use Base62 encoding (a-z, A-Z, 0-9)
- Example :
  ID in DB = 1000
  Base62(1000) = "G8"
  Short URL = short.ly/G8
- Ensures unique and compact codes.

6. Database Schema (Example)

Column	Type	Description
id	BIGINT (PK)	Auto increment ID
short_code	VARCHAR	Unique short code
long_url	TEXT	Original long URL
created_at	TIMESTAMP	Creation time
click_count	BIGINT	Total clicks

7. Scalability

- Use load balancer to distribute traffic.
- Cache frequent redirects in Redis.
- Database replication and sharding.
- Store analytics in separate service.
- CDN can be used for static content.

8. Key Components

- Web Server : Handles requests.
- Cache (Redis) : Stores mapping for fast redirect.
- Database : Stores URL mappings.
- Analytics Service : Tracks clicks and provides reports.
- Load Balancer : Distributes incoming traffic.

9. Extra Features

- Custom alias (e.g., short.ly/my-link)
- Expiry for links.
- QR code generation.
- Click analytics (location, device, time).
- Rate limiting to prevent abuse.

10. Example

- Long URL : https://www.example.com/articles/system-design
- Short URL : short.ly/Ab12
- User clicks short.ly/Ab12
  → Web Server checks cache
  → If found, redirect to original URL
  → If not, fetch from DB, add to cache, then redirect.

11. Possible Tech Stack

- Backend : Java / Python / Go / Node.js
- Database : MySQL / PostgreSQL
- Cache : Redis
- Load Balancer : Nginx / HAProvy
- Analytics : Kafka + ClickHouse / ElasticSearch